

Contents

guide Attention Is All You Need	1
0. 这篇论文到底在改写什么	1
1. 论文结构地图	1
2. 背景: 为什么 RNN 不够了	2
3. 总体架构: encoder-decoder 还是保留的	2
4. Scaled Dot-Product Attention	3
5. Multi-Head Attention: 为什么要分头	3
6. Position-wise FFN: attention 之后还要非线性加工	3
7. Positional Encoding: 没有 RNN 后怎么知道顺序	4
8. 为什么 self-attention 在复杂度上有吸引力	4
9. Training recipe: 不是只有架构	4
10. 实验结果怎么读	5
11. Table 3 ablation: 设计理由的证据	5
12. 这篇论文没有解决什么	6
13. 和本仓库代码怎么对上	6
14. 读完后你必须能闭卷讲出来	6
15. AI agent 学习方式	7

guide_Attention Is All You Need

原论文: Attention Is All You Need

本地原文 PDF: [learning/transformer-deep/paper/01_attention_is_all_you_need.pdf](#)

作者: Vaswani et al.

年份: 2017

类型: paper

0. 这篇论文到底在改写什么

这篇论文的核心主张很硬: 序列建模不一定需要 recurrence, 也不一定需要 convolution; 只用 attention, 再配上前馈网络、残差、归一化和位置编码, 就可以做出当时最强的机器翻译模型。

在 2017 年以前, 主流机器翻译系统多是 RNN/LSTM/GRU seq2seq, 再加 attention。RNN 的问题是时间步之间天然串行, 长序列训练慢, 而且远距离依赖要穿过很多 hidden state。CNN seq2seq 可以并行一些, 但长距离信息要经过多层卷积传播。Transformer 的故事就是: 如果每个 token 在一层里就能直接看见所有 token, 长距离路径长度会变短, 并行度会变高。

读这篇论文时要抓住三个变化:

1. 计算图从时间递归变成全局 attention 图。
2. 表示学习从单个 hidden state 变成多头子空间交互。
3. 序列顺序不再由 RNN 天然提供, 而是通过 positional encoding 显式加入。

1. 论文结构地图

原文大体可以按下面顺序读:

1. Introduction: 解释为什么 RNN/CNN 有并行和长依赖瓶颈。

2. Background: 承认已有 attention、memory network、ConvS2S 等工作,但指出本文的纯 attention 架构更直接。
3. Model Architecture: 论文主菜, 定义 encoder、decoder、scaled dot-product attention、multi-head attention、FFN、embedding、positional encoding。
4. Why Self-Attention: 从每层复杂度、可并行度、最长路径长度三个角度解释为什么 self-attention 值得替代 RNN/CNN。
5. Training: 给出 WMT 数据、batch、optimizer、learning rate schedule、regularization。
6. Results: 报告 WMT 2014 英德和英法翻译结果。
7. Ablation: 改 heads、key/value 维度、model size、dropout、positional encoding, 看 BLEU 和 perplexity 怎么变。
8. Conclusion: 强调 attention-only 架构的可扩展性, 并展望图像、音频、视频等模态。

如果你只看一遍, 优先精读第 3、4、5、6 节。第 3 节告诉你模型是什么, 第 4 节告诉你为什么这么设计, 第 5/6 节告诉你它不是玩具模型, 而是在真实翻译任务上打赢了强 baseline。

2. 背景: 为什么 RNN 不够了

机器翻译任务可以写成条件生成: 给定源句子 $x=(x_1,...,x_n)$, 生成目标句子 $y=(y_1,...,y_m)$ 。早期 seq2seq 用 encoder RNN 把源句子扫一遍, 再用 decoder RNN 一个 token 一个 token 生成。attention 机制后来解决了单个瓶颈向量不够的问题, 让 decoder 每一步可以看 encoder 各位置。

但 RNN 还有两个根本问题:

- 串行依赖: h_t 依赖 h_{t-1} , 训练时不能完全并行。
- 信息路径长: 第一个 token 影响最后一个 token 要经过 $O(n)$ 个 recurrent step。

CNN seq2seq 改善了并行, 但如果卷积核很小, 远距离 token 仍然要经过多层才能交互。Transformer 的 self-attention 让任意两个 token 在同一层里直接建立联系, 路径长度是 $O(1)$ 。这就是题目里 "Attention Is All You Need" 的真正含义: 不是说模型只有 attention, 而是说序列 token 之间的信息混合不必靠 RNN/CNN。

3. 总体架构: encoder-decoder 还是保留的

Transformer 仍然是 encoder-decoder 架构。

Encoder 负责读源句子。原文 base model 用 6 层 encoder, 每层有两个 sub-layer:

1. multi-head self-attention。
2. position-wise feed-forward network。

每个 sub-layer 外面都有 residual connection 和 layer normalization。也就是说每个子层不是直接替换输入, 而是学习一个增量:

$\text{LayerNorm}(x + \text{Sublayer}(x))$

Decoder 负责生成目标句子, 也有 6 层。每层有三个 sub-layer:

1. masked multi-head self-attention: 目标端只能看见当前位置之前的 token, 不能偷看未来答案。
2. encoder-decoder attention: decoder 的 query 去看 encoder 输出的 key/value。
3. position-wise feed-forward network。

这套结构后来成为几乎所有 decoder-only LLM 的祖先之一。GPT 系列去掉了 encoder-decoder attention, 只保留 masked self-attention + FFN; BERT 更像 encoder 堆叠; T5/BART 仍然是 encoder-decoder。

4. Scaled Dot-Product Attention

原文最关键公式是:

$$\text{Attention}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) V$$

把它拆开:

- Q 是 query, 表示 “我现在想找什么信息”。
- K 是 key, 表示 “每个位置可以被什么方式匹配”。
- V 是 value, 表示 “如果匹配上了, 真正拿走的信息是什么”。
- QK^T 得到每个 query 对每个 key 的相似度。
- softmax 把相似度变成权重。
- 权重乘 V 得到加权汇总的信息。

为什么要除以 $\sqrt{d_k}$? 如果 d_k 很大, 随机向量点积的方差会变大, softmax 输入绝对值容易变大, softmax 会饱和, 梯度变小。除以 $\sqrt{d_k}$ 是一个尺度校正, 让 attention logits 保持在更稳定的范围。

这一步的设计理由非常重要: Transformer 不是只发明了 “看所有 token”, 还认真处理了数值稳定性。后面的 RMSNorm、RoPE、FlashAttention、KV cache, 本质上也都在围绕 “同一个 attention 公式如何更稳定、更高效、更长上下文” 继续改。

5. Multi-Head Attention: 为什么要分头

单头 attention 会把所有信息压进一个相似度空间。Multi-head attention 的做法是把表示投影到多个子空间, 在每个 head 里分别做 attention, 然后 concat 回来:

$$\begin{aligned} \text{head}_i &= \text{Attention}(Q W_i^Q, K W_i^K, V W_i^V) \\ \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O \end{aligned}$$

原文 base model:

- $h = 8$ heads。
- $d_{\text{model}} = 512$ 。
- 每个 head 的 $d_k = d_v = 64$ 。

这样总计算量和单个 512 维 attention 类似, 但模型可以在不同 head 里学不同关系: 对齐、局部短语、语法依赖、长距离指代等。注意这不是论文硬性证明出来的语义分工, 而是一个合理设计假设, 再由实验和可视化支持。

Table 3 的 ablation 很关键: 作者在计算量大致保持的情况下改变 head 数。单头比最佳设置低约 0.9 BLEU; head 太多也会掉, 因为每个 head 的维度太小, 匹配能力不足。结论不是 “head 越多越好”, 而是 “多个子空间有用, 但每个子空间要保留足够容量”。

6. Position-wise FFN: attention 之后还要非线性加工

每层 attention 负责 token 之间的信息交换, 但每个位置拿到混合信息后, 还需要独立的非线性变换。原文用的是两层 MLP:

$$\text{FFN}(x) = \max(0, x W_1 + b_1) W_2 + b_2$$

base model 里:

- $d_{\text{model}} = 512$ 。
- $d_{\text{ff}} = 2048$ 。

所以 FFN 先把每个 token 的表示升到更宽的隐藏层，再投回 512。现代 LLM 里 FFN 往往占参数和计算的大头，并且 ReLU 后来常换成 GELU、SwiGLU、GEGLU 等。你本仓库 `swiglu.py` 看到的内容，就是这条线的后续演化。

7. Positional Encoding: 没有 RNN 后怎么知道顺序

self-attention 本身对 token 集合是 permutation-invariant 的。也就是说，如果不加入位置信息，模型无法天然知道谁在前谁在后。

原文加入 sinusoidal positional encoding:

```
PE(pos, 2i) = sin(pos / 10000^(2i / d_model))
PE(pos, 2i+1) = cos(pos / 10000^(2i / d_model))
```

直觉是：不同维度对应不同频率的正弦/余弦波，低频维度表示长距离变化，高频维度表示短距离变化。这样每个位置都有一个确定的向量，并且相对位移可以由线性关系表达。

论文也试了 learned positional embeddings, Table 3 里效果几乎一样。作者选择 sinusoidal 的理由是它可能外推到训练时没见过的更长序列。这个点后来发展出大量位置编码工作: RoPE、ALiBi、NTK-aware scaling、YaRN、LongRoPE。读长上下文专题时，你会不断回到这里。

8. 为什么 self-attention 在复杂度上有吸引力

原文第 4 节比较三件事:

1. 每层计算复杂度。
2. 可并行计算量。
3. 两个位置之间的最长路径长度。

对于序列长度 n 、表示维度 d :

- self-attention 每层大约 $O(n^2 d)$ 。
- recurrent layer 大约 $O(n d^2)$ 。
- convolution 取决于 kernel size 和层数。

当 n 小于 d 或接近常见翻译句长时，self-attention 不一定比 RNN 更贵，还更并行。更重要的是路径长度: self-attention 任意两个 token 一层可达，RNN 需要 $O(n)$ ，卷积需要 $O(\log n)$ 或 $O(n/k)$ 。这解释了为什么 Transformer 擅长建模长距离依赖。

但这也埋下后来的问题: $O(n^2)$ attention 在长上下文时会变成瓶颈。FlashAttention、sparse attention、sliding window、linear attention、Mamba 等工作，都是在回应这个代价。

9. Training recipe: 不是只有架构

论文的训练细节非常值得看，因为很多后来的 Transformer 训练习惯都从这里来。

数据:

- WMT 2014 English-German, 约 450 万句对。
- WMT 2014 English-French, 约 3600 万句对。
- 使用 byte-pair encoding, 把源/目标共享或近似共享的 subword vocabulary 控制在约 37000 tokens。

优化:

- Adam optimizer。

- $\text{beta1} = 0.9$ 。
- $\text{beta2} = 0.98$ 。
- $\text{epsilon} = 1\text{e-}9$ 。
- $\text{warmup_steps} = 4000$ 。

学习率 schedule:

```
lrate = d_model(-0.5) * min(step_num(-0.5), step_num * warmup_steps(-1.5))
```

这个 schedule 先线性 warmup，再按 step 的 inverse square root 衰减。warmup 的意义是训练早期参数和梯度尺度还不稳定，直接大步更新容易炸；warmup 给模型一个“慢启动”阶段。

正则化:

- dropout = 0.1。
- label smoothing = 0.1。

Label smoothing 会让模型不要把正确 token 的目标概率当成 1，而是留一点概率质量给其他 token。原文说它会让 perplexity 看起来变差，因为模型被训练得“不那么自信”，但 BLEU/accuracy 会更好。这是一个很好的提醒：训练 loss 最低不一定等于生成质量最好。

训练成本:

- base model: 8 张 NVIDIA P100，约 100,000 steps，约 12 小时。
- big model: step 更慢，约 300,000 steps，约 3.5 天。

这在当时是很强的工程论点：不只是效果好，而且训练成本比很多 RNN/CNN 强系统低。

10. 实验结果怎么读

主结果在 WMT 2014 翻译:

- English-to-German: Transformer big 达到 28.4 BLEU，比当时包括 ensemble 在内的最好结果高 2 BLEU 以上。
- English-to-French: 论文正文报告 big model 达到 41.0 BLEU；摘要中强调 single-model state-of-the-art 41.8 BLEU，且训练约 3.5 天。

读这个结果时不要只记“赢了”。要看证据链:

1. baseline 是当时强机器翻译系统，包括 ConvS2S、GNMT、ByteNet 等。
2. 比较维度不只有 BLEU，还有训练成本。
3. base model 已经很强，big model 进一步提升。
4. 论文没有说 attention 永远更便宜；它说在当时翻译长度和硬件条件下，attention-only 更并行、更有效。

这就是一篇架构论文最理想的证据形态：一个清楚的瓶颈，一个简洁的新结构，一个可复现的 training recipe，一个强 benchmark 结果，再加 ablation 解释设计不是拍脑袋。

11. Table 3 ablation: 设计理由的证据

Table 3 是你判断论文有没有认真验证设计的地方。

几个重点:

- Head 数: 单头会掉分，太多头也会掉，说明 multi-head 有用但不是越多越好。
- d_k 维度: key/query 维度太小会影响质量，说明 compatibility 计算需要足够表达能力。
- model size: 更大的 d_{model} 、 d_{ff} 、更多参数会提高 BLEU，但也增加训练成本。

- dropout: 正则化影响很明显, Transformer 并不是天然不需要 regularization。
- positional encoding: learned 与 sinusoidal 结果接近, 所以 sinusoidal 的优势主要是外推假设和简洁性, 不是因为在这个实验里大幅赢。

新手常犯的错是跳过 ablation。其实 ablation 才告诉你: 这篇论文的哪些设计是核心, 哪些只是当时的合理默认。

12. 这篇论文没有解决什么

Transformer 很强, 但原文没有解决后来 LLM 的所有问题:

- 长上下文 $O(n^2)$ 成本没有被解决。
- KV cache 和 serving 问题不是本文重点。
- decoder-only 大规模语言模型不是本文实验对象。
- 位置编码外推只是一个希望, 不是充分解决。
- 翻译 BLEU 不能代表所有生成任务。
- attention 可解释性只通过少量可视化展示, 不能过度解读成完整因果解释。

这些局限并不削弱论文地位, 反而解释了为什么后来会有 RoPE、FlashAttention、PagedAttention、MQA/GQA、SwiGLU、RMSNorm、MoE、long-context scaling 等一整串工作。

13. 和本仓库代码怎么对上

优先打开:

- learning/transformer-deep/lectures/01-transformer-recap.md
- learning/transformer-deep/src/mha.py
- learning/transformer-deep/src/pe_sinusoidal.py
- learning/transformer-deep/src/gpt_mini.py
- learning/transformer-deep/src/tests/test_gpt_mini_forward.py

建议实验:

1. 在 mha.py 里打印 Q/K/V shape, 确认 batch、heads、seq、head_dim 的变化。
2. 把 head 数从 8 改成 1 或 16, 看参数 shape 和输出是否仍然一致。
3. 在 pe_sinusoidal.py 里画不同维度的位置编码曲线, 理解高频/低频。
4. 在 gpt_mini.py 里看 causal mask, 理解 decoder 为什么不能看未来 token。
5. 跑 KV cache 测试, 理解原论文训练架构如何演化到现代自回归推理。

14. 读完后你必须能闭卷讲出来

你应该能不看笔记回答:

1. Transformer 为什么能比 RNN 更并行。
2. scaled dot-product attention 为什么要除以 $\sqrt{d_k}$ 。
3. multi-head attention 的收益和代价是什么。
4. encoder self-attention、decoder masked self-attention、encoder-decoder attention 三者有什么区别。
5. positional encoding 为什么必要, sinusoidal 的设计直觉是什么。
6. FFN 在每层里负责什么, 为什么 attention 后还要 MLP。
7. warmup learning rate 和 label smoothing 分别解决什么训练问题。
8. WMT 实验证明了什么, 没有证明什么。
9. $O(n^2)$ attention 后来为什么成为长上下文瓶颈。

15. AI agent 学习方式

不要让 agent 直接再总结一遍。你应该这样用:

我已经读完 Attention Is All You Need 的导读。请你用闭卷口试方式问我 10 个问题。

每次只问 1 个, 等我回答后指出错误。

问题必须覆盖: attention 公式、multi-head、position encoding、training recipe、Table 3 ablation、现

然后再让 agent 帮你做代码实验:

请基于 learning/transformer-deep/src/mha.py 设计一个最小实验:

比较 1 head、4 heads、8 heads 时 attention 输出 shape、参数量和在一个 toy loss 的变化。

先让我自己预测结果, 再给我运行代码。

这篇论文真正进脑子的标志不是你能背标题, 而是你能从零画出 attention 数据流, 并解释为什么现代 LLM 的大部分工程优化仍然围绕这个数据流转。